Data Augmentation in Property Graphs using agentic AI: measuring human effort, task difficulty, environmental footprint and accuracy[⋆,⋆⋆]

Felipe Vasconcelos[a,*], Cristina Aguiar[a], Alexandre Chanson[b], Léo Gaillard[b], Mirian Halfeld-Ferrari[c], Patrick Marcel[c] and Verónika Peralta[b]

[a] *ICMC, University of São Paulo, São Carlos, Brazil*
[b] *LIFAT, Université de Tours, France*
[c] *Université d'Orléans, INSA CVL, LIFO, UR 4022, Orléans, France*

ARTICLE INFO

Keywords:
Data augmentation
property graphs
LLM Agentic AI

ABSTRACT

TO BE CHANGED

This paper investigates the use of Large Language Models (LLMs) and Agentic Artificial Intelligence (Agentic AI)[Vero] to complete missing attribute values in property graphs. Addressing this problem requires leveraging both the structural context of a node and the attribute information already present in the graph. We explore the use of LLMs to infer or retrieve plausible attribute values from external sources and the use of Agentic AI to execute multi-step retrieve tasks[Vero]. We propose an experimental protocol to evaluate AI-based[Vero] data augmentation in terms of accuracy, prompting effort, and environmental impact, through graphs and retrieval tasks of varied difficulty.[Vero] TO BE UPDATED:[Vero] Our preliminary results show that most of the models tested struggle with augmentation tasks and that context is useful for difficult tasks, with variations in the model and prompting strategy used.

Here is a new proposal for structuring the paper (10.10.26)... discussed (16/10) with Cristina and Patrick.[Mirian] OK for me.[Vero]
I replaced the list by the table of contents. At the end of Tuesday morning, the structure will reflect our discussion about sections. The TOC is to delete at the end, of course.[Vero]

Contents

1 Introduction TO DO: Turn framework for completion to assessment (motivation, contributions); reuse all contributions for updates to previous version.[Vero] **2**

2 Related Work TO DO: Discuss new approaches (if any); add Agentic approaches. ALEX is searching[Vero] **3**

3 An algorithm/approach/framework[Vero] for Assessing LLM-Based Data/Property[Vero] Completion/Augmentation[Vero] **5**
3.1 General overview NEW CONTENTS; CAN BE READ[Vero] 5
3.2 Task Difficulty NEW VERSION (with issues to discuss)[Vero] 6
3.3 Task Generation 9
3.4 Evaluation metrics 9
3.4.1 Effort metric 9

⋆ This work was partially supported by JUNON Program (DATA/LIFO) with financial support of Région Centre-Val de Loire, France, and partially supported by project TopOL with financial support of the French National Research Agency (ANR).Mention to JUNON still valid?[Vero]

⋆⋆ The first and second authors were supported by the São Paulo Research Foundation (FAPESP), the Brazilian Federal Research Agency (CNPq), and the Coordenação de Aperfeiçoamento de Pessoal de Nível Superior, Brasil (CAPES), Finance Code 001.

* Corresponding author

felipevsc@usp.br (F. Vasconcelos); cdac@icmc.usp.br (C. Aguiar); Alexandre.Chanson@univ-tours.fr (A. Chanson); leo.gaillard@univ-tours.fr (L. Gaillard); mirian@univ-orleans.fr (M. Halfeld-Ferrari); Patrick.Marcel@univ-orleans.fr (P. Marcel); veronika.peralta@univ-tours.fr (V. Peralta)

ORCID(s): 0009-0007-7922-9746 (F. Vasconcelos); 0000-0002-7618-1405 (C. Aguiar); 0000-0001-9195-5950 (A. Chanson); 0000-0003-2601-3224 (M. Halfeld-Ferrari); 0000-0003-3171-1174 (P. Marcel); 0000-0002-9236-9088 (V. Peralta)

4 Description of models and strategies[Vero] 11
4.1 Model selection DOLAP contents; TO BE UPDATED[Vero] . . . 11
4.2 Prompt Engineering Techniques DOLAP contents; TO BE UPDATED[Vero] . . . 11
4.3 Search Agent Configuration and Integration TO WRITE[Vero] . . . 12
4.4 Task Execution Algorithms TO REWORK[Vero] . . . 12
4.4.1 Strategy A: Static Parametric Pathway . . . 12
4.4.2 Strategy B: Tool-Augmented Agentic Pathway . . . 13

5 Experiments 13
5.1 Datasets DOLAP contents BEING UPDATED BY LEO[Vero] . . . 13
5.1.1 The Airport dataset . . . 13
5.1.2 The Football dataset . . . 14
5.1.3 The Music dataset . . . 14
5.2 Implementation DOLAP contents TO UPDATE[Vero] . . . 14
5.3 Lessons learned DOLAP contents TO UPDATE[Vero] . . . 14

6 Conclusion DOLAP contents TO UPDATE[Vero] 16

1. Introduction TO DO: Turn framework for completion to assessment (motivation, contributions); reuse all contributions for updates to previous version.[Vero]

The vocabulary is not stable as will surely by criticized. It can even be confusing. We start speaking about nodes, relationships, and attributes (2nd sentence). Then, properties and edges appear, without comments. 1st sentence introduces graph databases, but from 3rd sentence we speak about property graphs. [Vero]

I propose to introduce real-world concepts (entities, relationships and attributes) but quickly speak about nodes, edges and properties.[Vero]

Graph databases have become an established paradigm for modeling complex, highly interconnected domain knowledge. Because real-world graphs are frequently constructed by integrating heterogeneous and asynchronous sources, they naturally evolve over time through the progressive addition of nodes, edges and properties, representing real-world entities,[Vero] relationships, and attributes. Consequently, many real-world property graphs suffer from data incompleteness—manifesting as missing property[Vero] values, missing properties (i.e. undefined property-value pairs resulting in[Vero] sparse properties), or missing edges (i.e.[Vero] undefined relationships resulting in less-connected graphs)[Vero]. This incompleteness substantially degrades the performance of downstream graph querying, analytical tasks, and reasoning algorithms. To mitigate graph sparsity without relying on manual curation, automated techniques for data generation, discovery, and attributeDELETE attribute[Vero] augmentation have emerged as promising mechanisms for graph completion [42]. While existing research has mainly focused on inferring or predicting missing edges, the completion of node property[Vero] values remains underexplored.

Addressing this problem requires considering both the structural context of a node and the properties[Vero] already available in the graph. In this paper, we assume a scenario in which, for each node, we construct retrieval tasks[Vero] that include: (1) a set G of existing ("property", "value") pairs, and (2) a set H of property names for which values are sought. The selection of properties included in G and H takes into account the popularity of nodes and properties[Vero] within the graph, allowing us to analyze different task difficulties.

In our previous work [33], to solve this problem, we proposed an approach for completing property[Vero] graphs with missing values for selected node attributes, exploring the use of Large Language Models (LLMs) to infer or retrieve plausible attribute values. This initial approach relied on a static input-to-output pipeline (Input → Prompt/Model → Output), leveraging the model's internal parametric knowledge alongside the graph-based notion of context introduced in [**?**]. Is-it the REF to arxiv preprint [12]?[Vero] While this static method effectively generates high-confidence candidate values for missing attributes, enforcing strict confidence prompts can lead to conservative model behavior—resulting in abstention and, consequently, lower recall.

As an alternative execution pathway aimed at mitigating the abstention rates of static pipelines, we introduce an Agentic Artificial Intelligence ([Vero]Agentic AI) architecture operating as a dedicated *Search Agent*.

Thus, in this paper, we propose a **general Graph Augmentation Framework** for Property Graph completion. Rather than viewing static prompting and external agentic search in isolation, our framework formalizes two distinct operational pathways:

1. **Strategy A (Static Parametric Pathway):** A pipeline relying strictly on the LLM's internal parametric memory and guided by contextual information [33].
2. **Strategy B (Single-Turn Agentic Pathway):** A lightweight, single-interaction Search Agent that combines the LLM's reasoning engine with an external retrieval tool (e.g., Tavily). Given known (attribute, value) pairs in G and target attributes H, the agent plans a retrieval strategy, decomposes intent into parallel or sequential sub-queries, executes a single-turn tool invocation, and synthesizes raw evidence into graph updates without introducing complex, multi-turn conversational overhead.

Through a rigorous empirical study, we evaluate and compare **Strategy A** and **Strategy B** across key performance dimensions.

Main contributions.

- **A General Graph Augmentation Framework** that structures Property Graph attribute completion into static or agentic operational modes through a general execution protocol, specialized according to the underlying search model used.
- **A lightweight, single-turn Search Agent (Strategy B)** capable of parallel query decomposition and sequential dependency resolution using an external web search tool (e.g., Tavily).
- **A new automated task construction algorithm** that generates structured tuples (n, G, H) of known context properties G and target missing attributes H for each node n in the property graph.
- **A thorough comparative analysis** between Strategies A and B,[Vero] evaluating performance across accuracy, prompting effort, and execution sustainability while refining the evaluation metrics from [33] for task difficulty and computational effort.
- **An expanded empirical benchmark** spanning three distinct property graph datasets, significantly broadening the single-dataset evaluation scope established in [33].

Paper organization. Section 2 discusses related work. Section 3 describes the proposed framework for property graph attribute completion, introducing tasks and metrics, and presenting Strategies A and B. Section **??** describes the experimental setup and discusses experimental results. Finally, Section 6 concludes the paper. [Vero].

2. Related Work TO DO: Discuss new approaches (if any); add Agentic approaches. ALEX is searching[Vero]

This is a new file. The old file is still in the directory, Here, I added some references and text... keeping the OLD text too[Mirian]

The landscape of digital information discovery and model augmentation has evolved rapidly. The advent of Large Language Models (LLMs) is rapidly transforming digital information seeking. Recent comprehensive surveys [38, 41] highlight the evolutionary shift from traditional keyword matching—which struggles with complex synthesis [41][—and static LLM chatbots or naive Retrieval-Augmented Generation (RAG) [38, 41] toward proactive, autonomous Search Agents [38]. These next-generation Deep Research systems combine autonomous reasoning, iterative search, and information synthesis [41]. As systematically analyzed in [38], search agent frameworks span multiple core dimensions: (1) *architectural search structures* (categorized into parallel, sequential, and hybrid trajectories) [38] (2) *optimization strategies* (encompassing tuning-free multi-agent/test-time scaling methods and tuning-based SFT and reinforcement learning) [38], and (3) *practical applications and evaluation protocols* (covering internal tool/memory enhancements, external domain-specific deployments, and evolving rule to agent-as-a-judge evaluation frameworks) [38]. Specifically regarding internal reasoning and tool integration, in [37] the authors propose frameworks that equip reasoning models with specialized agentic tools such as web-search agents, Python code execution, and graph-based memory structures like Mind-Maps to maintain coherence across long reasoning chains and complex multi-step research tasks.

While these advanced AI search paradigms primarily target exploratory information seeking (for which we do not have a single, definitive, or pre-determined correct answer), structured data domains have been dealing with parallel challenges known as *data discovery* and *data augmentation*, which are foundational tasks traditionally aimed at

identifying relevant datasets, enriching incomplete tables, and imputing missing values. This section aims at reviewing these areas as they represent strategies to complete missing values in a data set. We are also interested in considering studies on LLM sustainability.

Data Discovery and Augmentation. *Data discovery*, also called *dataset search* or *table search*, concerns the discovery, exploration and retrieval of datasets related to an information need [9]. It was initially developed for *data integration*, aiming to enrich a dataset with additional instances (unionable datasets) or attributes (joinable datasets). Data discovery is more complex than keyword-based search, as it must account for factors such as data provenance, annotations, quality, and content and schema granularity to assess a dataset's fitness for use [9]. Many work focus on identifying a relevant subset of data sources within an organization (typically stored in a data lake) that are semantically similar to a query table [14]. They propose varied strategies for building dataset profiles, which are then used to build indexes or knowledge graphs for efficient query evaluation [8, 7]. Other work explores Web search by combining information retrieval and Semantic Web techniques, including *entity search* and *Web table search* [9, 28]. Many systems build knowledge graphs interconnecting datasets and columns while capturing their metadata and semantics, offering structured (e.g. SPARQL) or natural language queries [19, 34]. Recent approaches leverage language models for data discovery tasks [15], encoding datasets which are then retrieved by comparing vector representations [11, 22]. AutoDDG [39] leverages LLMs to automatically enrich summaries with semantic information and produce human-readable descriptions. Recent methods for table search are largely embedding-based, and the use of LLMs for these tasks remains underexplored [15]. A limitation of all these systems is the need to build and maintain large data repositories, while our proposal leverages context and already trained models. Indeed LLM have been demonstrated to store facts in their feed forward layers [29] with contextual information playing a key role in retrieving facts from a model [17].[Alex]

Data augmentation (DA) expands and diversifies training datasets, reducing the need for new data collection and enriching models with synthetic examples. DA for Named Entity Recognition is surveyed in [20], emphasizing preservation of morphological, syntactic, and semantic properties. In contrast, [10] adopt a broader NLP view, framing LLM-based augmentation from data- and learning-centric perspectives. Both stress that synthetic data must satisfy task constraints rather than factual accuracy. Graph-based modeling is increasingly used for complex interactions, but sparse, noisy, and irregular data hinder learning. Graph-specific data augmentation (GDAug) improves generalization by modifying graph data, though graphs' non-Euclidean structure and interdependent topology and semantics make this challenging. GDAug techniques are surveyed in [42], proposing a taxonomy based on the hierarchy of graph elements. They cover heterogeneous, temporal, spatio-temporal graphs, and hypergraphs, and evaluate augmentation via downstream task performance and metrics assessing structural preservation and diversity. Although [42] ignores LLM-based augmentation, their evaluation criteria and methodology inspire us. LLMRec [35] uses LLMs to augment user–item graphs (inferring edges, enriching item attributes and profiling users) improving recommendation quality.

Sustainability of LLM. The widespread use of LLMs across domains raises growing concerns about their environmental and economic sustainability. The systematic review in [21] analyses LLM electricity consumption across two stages. While *training* is far more energy-intensive per run, it is infrequent; *inference*, executed continuously at scale, ultimately dominates cumulative electricity demand. The paper proposes a framework capturing distinct electricity-related challenges and emphasises the need for a holistic approach beyond the sole responsibility of developers. Inference-time energy usage is examined in [3]. The authors introduce EnergyMeter, an open-source tool that monitors CPU, GPU, memory, and storage energy with minimal overhead. Results show that software-level monitoring is feasible; latency alone does not predict energy use; model size increases latency and consumption sub-linearly due to parallelisation; and larger batch sizes reduce energy per token until GPU saturation. In [25], the authors study how linguistic complexity in prompts affects inference energy, showing that model architecture remains the dominant factor for efficiency, with prompt complexity playing a secondary but meaningful role. As explained in Section **??**, we use the Python library [30], that estimates energy use and hardware-related impacts through a bottom-up Life Cycle Assessment.

Measuring prompting effort and difficulty of LLM tasks The effort to prompt language models is often measured as an interaction cost, for instance by means of the time spent by human to craft the prompt or the number of refinements or clarification queries needed [40, 18, 32]. Recent works explicitly introduce a task difficulty metric for LLMs, mostly in two flavors, (i) regarding the model's output signals, or (ii) regarding external reference points. The first category can be thought of as defining a posteriori metrics to observe the model's results or internals while producing the result. For

instance, [13] express difficulty as the point at which adding more context (sentences) stops significantly lowering the model's error. [16] quantify difficulty based on the information required to solve the task. On the other hand, metrics can be based on external reference points, in an a priori fashion since this is computed before sending any prompt to the model. For instance, [23] use perplexity of the prompt as a proxy for how specific or vague a prompt is. [24] define difficulty the amount of time a qualified human expert would take to complete the task. [43] quantify the difficulty by analyzing the internal numerical representation of the input prompt.

Positioning of Our Work. In contrast to traditional data discovery and augmentation frameworks that rely on maintaining massive internal data lake repositories [14] or complex graph structural manipulations [42], our work investigates a more direct pathway for Property Graph attribute completion. Rather than retrieving joinable tables from local corpora, we explore how models can fulfill missing attribute information externally or statically through the model's internal knowledge. To this end, while avoiding recent trends that rely on complex multi-agent architectures or heavy multi-turn orchestration, we propose a lightweight, protocol-driven framework utilizing LLMs in two complementary flavors: a static zero-shot parametric baseline (*Strategy A*) and a lightweight, single-turn Search Agent (*Strategy B*) leveraging live external web search through tools like the Tavily API. Rather than introducing repository maintenance overhead or workflow complexity, our primary objective is to rigorously compare these strategies—benchmarking them across completion accuracy, prompting effort, and execution sustainability under varying structural graph constraints.

Perhaps add some comments concerning effort and Sustainability to this last paragraph ??Mirian

3. An algorithm/approach/frameworkVero for Assessing LLM-Based Data/PropertyVero Completion/AugmentationVero

This section describes our proposal for assessing LLM-based data augmentation strategies. We first present an overview of our framework and then describe augmentation tasks and evaluation metrics. The compared strategies are detailed in next section.Vero

3.1. General overview NEW CONTENTS; CAN BE READVero

The goal of our framework is to give augmentation tasks to several models (LLMs under several strategies) and compare their answers with a variety of metrics.

An augmentation task for a model is to predict the values of node properties, hidden from it.

Definition (Task). A task is a tuple (n, G, H) where n is a node, H is the set of properties for which we seek to predict values and G is the set of properties with their values, known by the model.

The models are given a set of (property,value) pairs (those in G), and are asked to infer the most likely real-world entity corresponding to such property values and retrieve trustworthy, recent values for the unknown properties (those in H). For example, given $G = \{(latitude, 48.857605), (longitude, 2.350295), (name, Paris)\}$, search values for $H = \{\text{population, cost of living index}\}$.

The workflow of our proposal is sketched in Figure 1. It operates in two distinct phases: task preparation/generationVero and evaluation/assessment/scoringVero. In the former, some structural statistics (centrality, context, and property popularity) are computed for each node of a property graph and used to build augmentation tasks of varied difficulty (detailed in Subsections 3.2 and 3.3). In the latter, LLMs are asked to search for hidden property values; their results are assessed according to accuracy, effort and energy impact/environmental impact/sustainabilityVero metrics (detailed in Subsection 3.4).

The protocol is formalized in Algorithm 1. Lines 1-7 deal with structural analysis of the property graph and task generation. For each node n, centrality $cent(n)$ and context (in particular neighbors) $context(n)$ are computed (Line 2). The popularity $pop(p)$ of each property p in node context are also computed (Line 4). Several tasks are generated sampling nodes and properties (Line 7, see also Algorithm 2). Lines 8-17 deal with task execution and scoring. For each model, strategy and task, we call the LLM (Line 12), assess metrics (Line 13) and arrange the result to be returned (Line 14).

The protocol is strategy-agnostic: for each task (n, G, H), it dispatches the input to several operational pathways (called strategies) which invoque LLM under several templates and agents. Such strategies are thus compared under identical graph conditions.

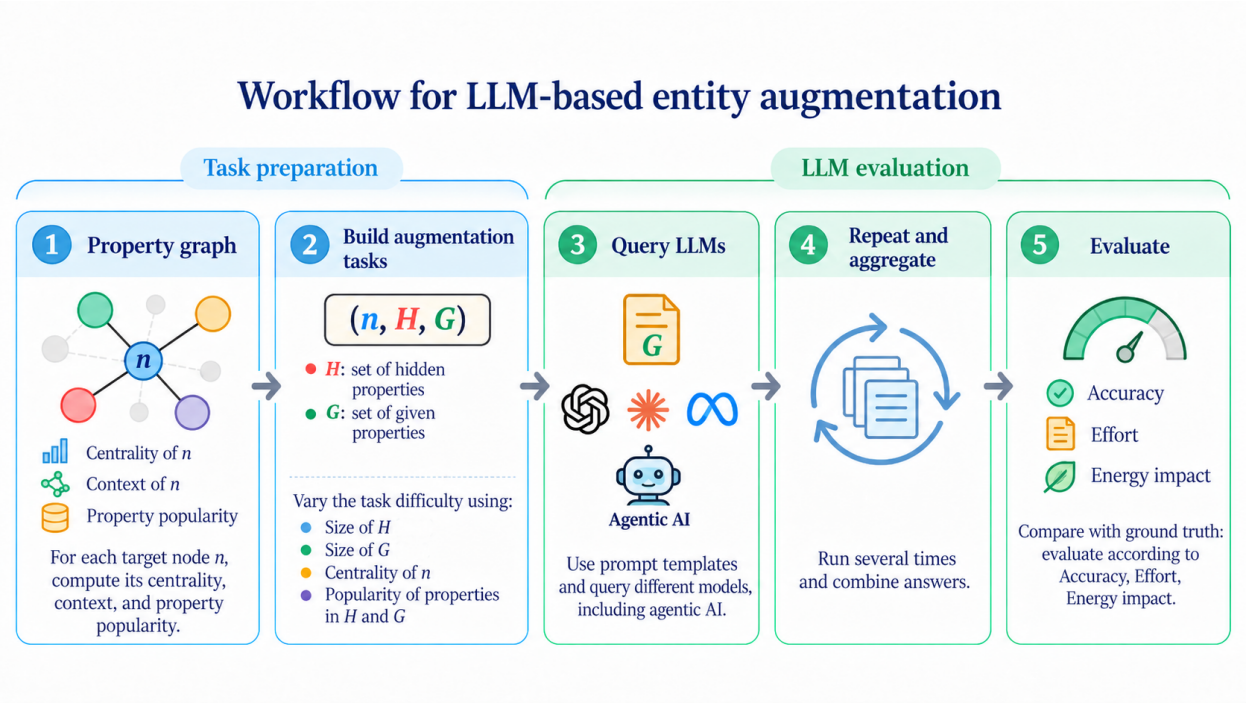


Figure 1: Overview of our pipeline

I like the spirit of the figure but I would change (i) colors, which seem to be random. I propose to use blue for node-related things, keep red and green for H and G, and yellow for properties. Then, in Step 1, it could be nice to use other colors for neighbour nodes. (ii) LLM likes to put the repeat in a separate step. IMHO, run several times is part of Step 3. Step 4 can be Aggregate, or just disappear (aggregate being part of Evaluate). I vote for disappear.[Vero]

WHAT I CHANGED IN THE ALGORITHM:

- strategy as an input
- task generation parameters r, nN, nH, nG as input (they are necessary to call Algo 2). nN, nG, nH replace max_n, max_G, max_H
- Replacement of task definition by task generation
- Replacement of (n, G_n, H_n) by (n, G, H)
- It is stupid to generate quartiles here (no change). It is better done in Algo 2. So, change in signature of Algo 2.
- iteration on strategies and general call instead of double call
- Add an state for assessment

There are still ugly global variables. [Vero]

3.2. Task Difficulty NEW VERSION (with issues to discuss)[Vero]

We posit that the difficulty of the task varies with[Vero] all three, the node n itself,[Vero] the characteristics of the properties to predict, i.e., H, and the information passed to the model, i.e., G, that include some properties of n[Vero] and some properties in its context.

Given a task (n, G, H),[Vero] we consider the following assumptions:

1. the more properties in H and the less in G (including properties from the node's context), the more difficult the task,

2. the more central n, the easier its properties in H will be to predict, and the less difficult the task. For instance, if airports are ranked by node centrality (using e.g., page rank), then airports of high centrality likely correspond to big hubs whose properties should be easier to retrieve by the model,

Algorithm 1 General Evaluation Protocol

Require: Set of models M, set of strategies S, set of nodes from property graph PG, task generation parameters nN, nG, nH, number of run iterations r

Ensure: Evaluation results $Result$ mapping models, strategies and[Vero] tasks to performance metrics (accuracy, effort, sustainability)

// Phase 1: Structural Analysis & Task Generation

for each node $n \in N$ **do**
  Compute node centrality $cent(n)$ and context $context(n)$
  for each property p found in $context(n)$ **do** FORMALIZE CONTEXT[Vero]
    Compute the popularity $pop(p)$ of p
  end for
end for

cent, context and pop are calculated but not passed to task generation. They are used as global variables, which is ugly[Vero]

$Tasks \leftarrow taskGeneration(N, nN, nG, nH)$ ▷ Generates tuples (n, G, H); see Algorithm 2

// Phase 2: Dual-Pathway Execution & Scoring

$Result \leftarrow \emptyset$
for each model $m \in M$ **do**
  for each strategy $s \in S$ **do**
    for each task $(n, G, H) \in Tasks$ **do**
      $res \leftarrow ExecuteModel(m, s, n, G, H, r)$ ▷ Call LLM; see Algorithm **??**
      $(acc,effort,sust) \leftarrow Assess\ (n, G, H, res)$ ▷ Assess accuracy, effort and sustainability
      $Result \leftarrow Result \cup \{\big(m, s, (n, G, H), res, (\ acc,effort,sust)\big)\}$
    end for
  end for
end for
return $Result$

3. the more popular the property in H (in terms of presence in the graph, i.e., the probability that the property has a value when observing a node of a given type), the easier to predict and the less difficult the task,
4. the more popular the property in G, the more likely it can appear in the training examples of the model, and consequently the easier for the model to use it, and the less difficult the task.
5. the higher the centrality of a property's element (node or arc) of n's context in G, the easier it is for the model to use it and the less difficult the task (e.g., if the airport city name of some highly central city is in G, it will be easier for the model to retrieve the airport's hidden properties),

These assumptions guide our formalization of task difficulty. To construct this measure, we first define the context of a node -as properties in G may come from neighbor nodes- and introduce metrics for centrality and popularity.[Vero]

DISCUSSION ON CONTEXT: In node comparison insights paper (the one cited), the notion of context is formally defined on a graph type S. By abuse of language, we sometimes refer to the context of a node n in G when we actually mean the context of the corresponding node nS in the graph type S. To determine the context of a node nS in a graph type S, we proceed as follows: 1) Consider all relationships incident to the corresponding node nS in S. 2) Consider the hierarchical environment of nS in S. Concretely, this means collecting all paths nS -> nS', and adding every element that appears on it to the context of n. Formally, the context of a node is a subgraph S1 S. An instance in G is a subgraph G1 G, that is valid with respect to S1.

Here, we should make easier, of course. We can still speak of a subgraph, but as we only want direct neighbors TO BE CONFIRMED and N-1 paths TO BE CONFIRMED, I propose to say that context is a set of (e,p), e being n union neighbourg elements along N-1 edges and p being their properties.

ARE YOU OK WITH THIS ?

IF YES, I CAN WRITE DEFINITIONS AND PATCH ALGORITHMS IN THIS WAY. BUT I WAIT. [Vero]

Definition (Context). The context of a node n is a set of properties coming from n, its direct neighbor nodes, the relationships to its direct neighbor nodes, and the properties along paths where relationships have a $n : 1$ cardinality,

starting from n (like e.g., relationship IN between a city and a country since a city is IN only one country). A formal definition of context is given in [1]. It is computed with function *context*().

Difficulty must also account for the *centrality* of n (Assumption 2) and the elements in its *context* (Assumption 5). Why this sentence is here?[Vero]

Definition (Centrality of an element). The popularity of an element (node or edge) e is given by a centrality measure $cent(e)$, assumed normalized, for this element CHECK NORMALISATION[Vero]. Alternatively, centrality can be replaced with, when randomly drawing a node in the graph, the probability of the node being source or target of the relationship.

Definition (Popularity of a property). Given a property p of an element (node or edge) e of type t, we call $pop(p)$ the popularity of p, defined as the probability of observing a (non null) value for property p when drawing an element in the set of elements of type t. This gives a probability distribution P for random variable p with two outcomes: $P(p \neq null)$ and $P(p = null)$. Note that we do not consider the probability distribution over the values of properties because the majority of values will be specific to the element and be present only once (e.g., latitude, name, length of runways, etc.).

Equipped with these measures, we can now formalize the *difficulty* of a task.

Definition (Difficulty of a task). Let $T = (n, G, H)$[Vero] be a task. $cent(n)$ is the centrality of n, assumed normalized. Given a property p, $pop(p)$ is the popularity of p, i.e., the probability of appearance of p among the set of all properties. $cent^p(e)$ is the centrality of the element e the property p comes from. The difficulty of task T is the geometric mean of the probability of seeing a property from H, the surprise brought by the centrality of n and the surprise brought by the popularity (weighted by centrality of their origin) of the properties in H and G:

$$\text{Difficulty}(T) = \sqrt[3]{\frac{|H|}{|H|+|G|} \times -log(cent(n)) \times \sum_{p \in H \cup G} -log((pop(p) \times cent^p(e)))}$$

In the above definition, to account for the respective sizes of H and G (Assumption[Vero] 1), we use the probability of drawing a property $p \in H$ from $H \cup G$, defined as $\Pr(p \in H) = \frac{|H|}{|H|+|G|}$. Note that $\Pr(p \in H) = 0$ when $H = \emptyset$ since there are no properties for which we need to find a value, while $\Pr(p \in H) = 1$ when $G = \emptyset$ since no properties with known values are provided to help. The edge case $H = G = \emptyset$ is undefined. Furthermore, to account for the centrality of node n (Assumption 2)[Vero], we measure the surprise introduced by its centrality, such as $-\log(\text{cent}(n))$. The task difficulty should also reflect the centrality of elements within n's context whenever context properties are present in G (Assumption 5)[Vero]. Finally, to account for Assumptions 3 and 4, we incorporate the surprise associated with the popularity of properties in H and G, weighted by the centrality of their respective element of origin (node or edge).

@Patrick: do you have a reference to justify the geometric mean?[Vero]

TO DISCUSS: I would like to say, somewhere, that G and H are not empty, and that G contains at least one property of n.[Vero]

Example I PROPOSE TO UPDATE THE EXAMPLE IN ORDER TO AVOID THE CASE DOES NOT CONTAIN ANY PROPERTY OF n. THIS MEANS, FOR EXAMPLE, ADDING OR REPLACING PROPERTIES FOR 2nd AND 3rd TASKS.[Vero] Table 1 illustrates the difficulty of various tasks. Table 1 (left) lists various properties p coming from different nodes[Vero] e, with the centrality of e and the popularity of p. Table 1 (right) lists various tasks built with these properties, and details the computation of their difficulty. It can be observed that, as desired, tasks with a central node and popular properties, like (e_1, H_1, G_1), have a low difficulty while tasks with a node of lower centrality and less popular properties, like $(e_3, H4, G4)$, have a much higher difficulty. Desirably, medium easy tasks like $(e_2, H2, G2)$, where H has popular (easy to predict) properties and G has less popular properties, are scored less difficult than tasks like $(e_2, H3, G3)$ where H has unpopular properties even if G has popular ones.

Important: is this desirable?[Patrick] Note that, two tasks $T_1 = (n_1, X, Y), T_2 = (n_2, Y, X)$, i.e., where the hidden of T_1 is the given of T_2 and vice versa, such that $|X| = |Y|$ and $cent(n_1) = cent(n_2)$, have the same difficulty.

@Patrick: In rewriting the definitions of P_i below... I think that, to make the subsets clear, the boundaries should appear. I kept them[Mirian]

A SIDE-EFFECT TO DISCUSS: In the second and third tasks, all given properties are of the context, none of the node itself. This is really difficult and is not captured by our definition of difficulty. As an exemple, we ask for an airport name, given the population and

Node[Vero]		Property	
e	$cent(e)$	p	$pop(p)$
e_1	0.7	a	0.6
e_1	0.7	b	0.4
e_2	0.4	c	0.1
e_2	0.4	d	0.3
e_2	0.4	e	0.6
e_3	0.2	f	0.2
e_3	0.2	g	0.1
e_3	0.2	h	0.2

Node	Task H and G	Intuitive difficulty	$\frac{\lvert H\rvert}{\lvert H\rvert+\lvert G\rvert}$	$-log(cent(n))$	$\sum_{p\in H\cup G}$	Difficulty
e_1	H1 = $\{b\}$ G1 = $\{a,d\}$	easy	0.33	0.51	6.14	1.01
e_2	H2 = $\{e\}$ G2 =$\{f,h\}$	medium easy	0.33	1.32	11.34	1.71
e_2	H3 = $\{c,d\}$ G3 = $\{b\}$	medium hard	0.66	1.32	9.54	2.03
e_3	H4 = $\{f,h\}$ G4 = $\{g\}$	hard	0.66	2.32	14.93	2.84

Table 1
Example of various tasks and their difficulty.

name of the country. In our experiments for "adding" context, we should be careful of not "replacing" node properties by context ones.[Vero]

LESS SERIOUS, but also evidenced by examples, the 3rd term, beign a sum, has higher values, biasing the product.[Vero]

3.3. Task Generation

Building upon the notion of task difficulty, our objective is to generate a set of tasks of diverse difficulty. To achieve this, Algorithm 2 constructs tasks using the following quartile subsets: ATTENTION: There are two algos labeled the same (algos 2 and 6).[Vero]

Let N be a set of nodes, $E = N \cup \text{context}(N)$, and let the set of property-score pairs be:

$$\mathcal{P} = \{(p, m_p) \mid e \in E,\ p \in \text{properties}(e),\ m_p = \text{pop}(p) \times \text{cent}^p(e)\}.$$

Context is informally defined as a set of properties. So, $E = N \cup \text{context}(N)$ has little sense. In addition, previously, e is used for elements (nodes and edges). In top of this, most people may think on E being edges as in classical graph definitions. The one that wrote this sentence seems to consider that context is the set of neighbourg nodes and edges, while the one that defined context, just focused on the properties. We should solve this.[Vero]

We partition $\mathcal{P}$ into four quartile subsets $\mathcal{P}_1, \ldots, \mathcal{P}_4$ using boundary limits $l_0, \ldots, l_4$, where l_1, l_2, l_3 represent the 25th percentile (lower quartile), 50th percentile (median), and 75th percentile (upper quartile), respectively:

$$\mathcal{P}_i = \{(p, m_p) \in \mathcal{P} \mid m_p \in [l_{i-1}, l_i)\} \quad \text{for } i \in \{1, 2, 3\},$$
$$\mathcal{P}_4 = \{(p, m_p) \in \mathcal{P} \mid m_p \in [l_3, l_4]\}.$$

Similarly, for the node centralities, let $\mathcal{C} = \{\text{cent}(n) \mid n \in N\}$ be the set of centrality scores for all nodes in N. Using boundary limits $q_0, \ldots, q_4$ corresponding to the 25th, 50th, and 75th percentiles of $\mathcal{C}$, we define the quartile intervals Q_1, Q_2, Q_3, Q_4 as:

$$Q_i = [q_{i-1}, q_i) \quad \text{for } i \in \{1, 2, 3\},$$
$$Q_4 = [q_3, q_4].$$

Algorithm 2 builds tasks by selecting a node n whose centrality falls within Q_i. Specifically, on line 8, the algorithm draws a set of target properties H of size $size_H$ whose score pairs belong to quartile partition $\mathcal{P}_j$. Concurrently, on line 11, it draws a set of given properties G of size $size_G$ whose score pairs belong to $\mathcal{P}_k$. The variation in task difficulty is systematically achieved by iterating over different combinations of node centrality tiers (Q_i), property score partitions ($\mathcal{P}_j$ and $\mathcal{P}_k$), and subset sizes ($size_H$ and $size_G$). For instance, given properties p_1, p_2, p_3 meeting the respective quartile criteria, the size loops generate varied task structures such as $(n, \{p_1\}, \{p_2\})$ (when $size_H = 1$ and $size_G = 1$) and $(n, \{p_1\}, \{p_2, p_3\})$ (when $size_H = 1$ and $size_G = 2$). Note that H and G are constructed to be mutually exclusive, as the algorithm explicitly samples G from the remaining properties ($\text{properties}(n \cup \text{context}(n)) \setminus H$), thereby ensuring that $G \cap H = \emptyset$.

3.4. Evaluation metrics

3.4.1. Effort metric

What about a sort of effort compression ratio: the total number of tokens humanly generated (by prompting, interacting with the agent) divided by the number of tokens AI generated (by the LLM or the agent)? Or its inverse, token amplification factor.[Patrick]

Algorithm 2 Task Definition Generation

Require: A set of nodes N and their contexts; integers max_n, max_H, max_G; pre-computed quartile partitions $\mathcal{P}_1, \ldots, \mathcal{P}_4$ and $Q_1, \ldots, Q_4$.
Ensure: A set of $27 \times max_n \times max_H \times max_G$ tasks varying in terms of difficulty.

$TASKS \leftarrow \emptyset$
for $Q_i \in \{Q_1,\ Q_2 \cup Q_3,\ Q_4\}$ **do**
  $NODES \leftarrow \text{Sample}(\{n \in N \mid \text{cent}(n) \in Q_i\},\ max_n)$ ▷ Draw up to max_n nodes from N
  for $n \in NODES$ **do**
    for $size_H \in [1, max_H]$ **do**
      for $size_G \in [1, max_G]$ **do**
        for $\mathcal{P}_j \in \{\mathcal{P}_1,\ \mathcal{P}_2 \cup \mathcal{P}_3,\ \mathcal{P}_4\}$ **do**
          $H \leftarrow \text{Sample}\left(\{p \in \text{properties}(n) \mid (p, m_p) \in \mathcal{P}_j\},\ size_H\right)$
          ▷ Draw $size_H$ properties of node n
          for $\mathcal{P}_k \in \{\mathcal{P}_1,\ \mathcal{P}_2 \cup \mathcal{P}_3,\ \mathcal{P}_4\}$ **do**
            $G \leftarrow \text{Sample}\left(\{p \in \text{properties}(n \cup \text{context}(n)) \setminus H \mid (p, m_p) \in \mathcal{P}_k\},\ size_G\right)$
            ▷ Draw $size_G$ properties not in H
            $TASKS \leftarrow TASKS \cup \{(n, H, G)\}$
          end for
        end for
      end for
    end for
  end for
end for
return $TASKS$

enviromental impact score for agents: wip

The energy impact score of each agent is estimated using the agentic energy multiplier proposed in the paper [4]:

$$E_{\text{agent}} = \left(N_{\text{calls}} + N_{\text{tools}} \cdot w_{\text{tool}}\right) \cdot E_{\text{single}},$$

where N_{calls} denotes the number of LLM invocations, N_{tools} the number of tool calls, w_{tool} a weight representing the relative energy cost of tool execution, and E_{single} the estimated energy consumption of a single LLM invocation, obtained using an energy estimation tool such as EcoLogits. The tool weight reflects computational complexity, with higher values assigned to more energy-intensive operations. For the LLM-only strategy, we set $N_{\text{calls}} = 1$ and $N_{\text{tools}} = 0$, allowing its energy consumption to be estimated using the same formulation and enabling comparison across strategies. Felipe & Cristina

Some quick thoughts after a first reading. I have the impression that this effort economy metric deviates from the effort metric of DOLAP. In DOLAP, effort was meant to measure the user effort in prompting the model, while here it is more related to the work done by the model. Besides, since the proposed metric incorporates difficulty, it is now inversely correlated with it. And because it is related to the work done by the model, it is likely correlated to the "sustainability" metric as well. I think we should avoid having 3 different measures that are correlated with one another. Maybe the solution is to devise one global metric? Patrick

wip – user effort. im planning on using Keystroke level model from [6]. the idea is to breakdown tasks into atomic/unit tasks. then, each unit task is associated with an operator that the authors assign the time the users take to perform that operation. these are the operators: K for pressing a keystroke, P for pointing with the mouse, H for homing the hands on the keyboard and D for drawing. so, the idea is to decompose each strategy into these operators.

LLM-only strategy. The user formulates a prompt requesting the airport property from the LLM's internal knowledge and submits it. The estimated user effort is:

$$E_{\text{LLM}} = N_K K + N_P P + N_H H + N_M M,$$

where N_K, N_P, N_H, and N_M denote the number of keystrokes, pointing actions, homing actions, and mental preparations, respectively. as we are still using keystrokes, we are able to correlate the original effort (prompt size) with this new metric.

Search-agent strategy. The user formulates a request for an AI agent to retrieve the airport property through web search and submits it. The agent performs the search and information extraction autonomously; therefore, these operations do not contribute to user effort. The estimated effort is:

$$E_{\text{agent}} = N_K K + N_P P + N_H H + N_M M.$$

Human web-search strategy. I dont know if we need this..t

The user searches the web manually, inspects search results, opens relevant pages, and extracts the requested property. The estimated effort is:

$$E_{\text{human}} = N_K K + N_P P + N_H H + N_M M + V,$$

where V represents the time spent visually searching for, reading, and extracting the requested information from webpages. V is supposed to be the time the user spends reading the results.

Felipe & Cristina

While this is indeed interesting, it may be quite hard to get all the information, in particular mouse pointing, etc. I think relying only on the input tokens (or in combination with tokens used/generated like I propose above) is a reasonable proxy.Patrick

I believe we can simply state that only one click is required (to submit the question to the LLM), or one click on a search result link, and so on. I think these are basic assumptions we can make. We don't need to know how long the user takes to click or type; we can use the baseline values defined in the keystroke-level model paper. However, I think the input tokens proposal could also be interesting.Felipe & Cristina

4. Description of models and strategiesVero

4.1. Model selection DOLAP contents; TO BE UPDATEDVero

We selected state-of-the-art proprietary large language models based on their performance in benchmarks such as LMArena[1] and Livebench [36]. We also selected smaller versions of these models to assess whether reduced-capacity models can achieve comparable performance with lower environmental cost. We chose proprietary models over open source models since the former are significantly more efficient and most commonly used on AI platforms [26].Felipe & Cristina

We evaluate Gemini-2.5-Pro, Gemini-2.5-Flash, GPT-5, GPT-5-mini, Claude-4.5-Sonnet, and Claude-4.5-Haiku. We exclude Gemini-3-Pro, as it does not provide a smaller counterpart, and Claude Opus, following Anthropic's recommendation to use Sonnet for general-purpose tasks [2]. All models are evaluated through their official APIs using identical prompts and default configuration settings. We did not use any form of retrieval-augmented generation (RAG). Instead, as explained in Section **??**, we rely on explicitly provided structured context, extracted from the graph.Cristina

4.2. Prompt Engineering Techniques DOLAP contents; TO BE UPDATEDVero

Prompt engineering refers to designing system instructions to guide the models for performing a specific task, without modifying its underlying parameters or weights [31]. Prompt engineering can improve results and perform similarly to fine-tuning without the cost of retraining the model [5]. Different techniques are available for performing prompt engineering, such as AutoPrompt, Tree of Thoughts, and Self-Consistency [31]. In this work, we focus on the most common ones: zero-shot, few-shot (1,3,5), and Chain-of-Thought (CoT). We also test a smaller, compact version of the base zero-shot prompt.

Figure 2 (left) summarizes the evaluated prompt techniques and their corresponding token lengths. For each prompt technique, we compute the number of tokens using the token-counting functions provided by the official APIs of the respective models and report the average token count across runs. The base prompt (zero-shot) has an average length of 219 tokens, whereas the compact variant contains 103 tokens. Prior work suggests that shorter prompts can lead to reduced computational overhead and energy consumption during inference, particularly for models that are sensitive to input length [25].

Our base prompt follows a zero-shot setting without CoT, in which the model is instructed to act as an assistant that enriches entities extracted from a database. Specifically, the model is asked to identify the most likely real-world entity corresponding to the input and to retrieve recent and trustworthy values for a predefined set of properties. The output is constrained to a CSV format to facilitate downstream processing and automated evaluation. Following prior work, we employ structured tags in the prompt, as this has been shown to improve response consistency and extraction quality. The user input consists of the properties extracted from the target node, augmented with contextual information (Section **??**). This mechanism resembles a RAG setup, but avoids the need for vector representations or similarity search, relying instead on explicitly provided structured contextCristina.

[1] https://lmarena.ai/leaderboard

Prompt technique	Tokens#
Compact	103
Zero-shot (Base)	219
1-Shot	258
Zero-shot + CoT	273
1-Shot + CoT	315
3-Shot	325
3-Shot + CoT	377
5-Shot	414
5-Shot + CoT	468

Models tested
Gemini-2.5-Pro
Gemini-2.5-Flash
GPT-5
GPT-5-mini
Claude-4.5-Sonnet
Claude-4.5-Haiku

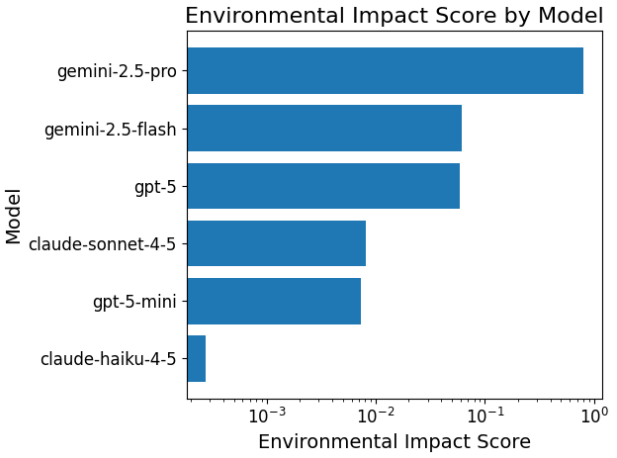


Figure 2: Prompting technique, ranked by formulation effort (mean number of tokens), models tested and Composite Environmental Impact Score (EIS) by Model

Algorithm 3 SimpleLLMRoutine (Strategy A)

Require: Model m, task (n, G_n, H_n), set of prompt templates T, number of run iterations $r \in \mathbb{N}_{\text{odd}}$
Ensure: Set $TemplateResults$ containing candidate completions aggregated across r runs for each template $t \in T$

1: $TemplateResults \leftarrow \emptyset$
2: **for** each prompt template $t \in T$ **do**
3: $\quad e \leftarrow Instantiate(t, n, G_n, H_n)$ ▷ Inject task into template t
4: $\quad Runs \leftarrow \emptyset$
5: $\quad$ **for** $k \leftarrow 1$ *to* r **do** ▷ Sampling across r runs
6: $\quad\quad res \leftarrow PromptModel(m, e)$ ▷ Zero-shot parametric inference
7: $\quad\quad Runs \leftarrow Runs \cup \{res\}$
8: $\quad$ **end for**
9: $\quad R_{m,e} \leftarrow Aggregate(Runs, m, e)$ ▷ PUT A COMMENT HERE. Majority voting ??[Mirian]
10: $\quad TemplateResults \leftarrow TemplateResults \cup \{(t, R_{m,e})\}$
11: **end for**
12: **return** $TemplateResults$

4.3. Search Agent Configuration and Integration TO WRITE[Vero]

4.4. Task Execution Algorithms TO REWORK[Vero]

4.4.1. Strategy A: Static Parametric Pathway

Strategy A implements the static, memory-bound methodology introduced in [33], as illustrated in Figure **??**Figure from Felipe's slides[Mirian]. In this approach, we leverage the internal parametric knowledge of Large Language Models (LLMs) grounded in known graph context to infer or complete missing property[Vero] values. The execution flow is formalized in the *SimpleLLMRoutine* procedure (see line **??** of Algorithm 1 and Algorithm 3). This is confusing. We should only reference Algo 3.[Vero]

Given a task (n, G_n, H_n), Algorithm 3 builds a zero-shot prompt template. The request is dispatched directly to model m, relying strictly on its parametric memory without invoking external APIs. To account for generation variance across non-deterministic LLM runs, each prompt template is executed multiple times (Algorithm 3, lines 5–8). Finally, the resulting candidate completions are aggregated across runs to produce the final attribute assignments (Line 9). To account for generation non-determinism across independent sampling runs ($r \in \mathbb{N}_{\text{odd}}$), the *Aggregate* function applies majority voting over extracted attribute values. For a given task (n, G_n, H_n) and template t, the value appearing in the majority of r runs is retained as $R_{m,e}$. Is this correct??[Mirian]

Algorithm 4 AgenticAIRoutine (Strategy B)

Require: Model m, benchmark task (n, G_n, H_n), sampling runs $r \in \mathbb{N}_{\text{odd}}$
Ensure: Aggregated candidate attribute completions R_m

1: $Runs \leftarrow \emptyset$
2: **for** $k \leftarrow 1$ *to* r **do** ▷ Multi-run sampling for agentic execution stability
3: $res \leftarrow ExecuteSearchAgent(m, n, G_n, H_n)$ ▷ Query decomposition & external retrieval (e.g., Tavily)
4: $Runs \leftarrow Runs \cup \{res\}$
5: **end for**
6: $R_m \leftarrow Aggregate(Runs, m)$ ▷ Consolidates multi-run agentic responses
7: **return** R_m

4.4.2. Strategy B: Tool-Augmented Agentic Pathway

Strategy B implements a tool-augmented operational pathway where the language model functions as an autonomous, single-turn Search Agent, combining its internal reasoning engine with external retrieval tools.

As illustrated in Figure **??**Felipe's slides[Mirian], given a target node and its missing graph attributes (n, G_n, H_n), the LLM formulates a query plan and decomposes its intent into targeted search queries. These queries are executed via an external retrieval tool (e.g., the Tavily Search API). Once external search snippets are retrieved, the LLM synthesizes and structures the results to construct the finalized attribute completions.

The execution of Strategy B is formalized in Algorithm 4. To handle generation non-determinism and agentic variance, the routine executes across r independent iterations (loop starting on line 2). In each run, the model invokes the search agent via *ExecuteSearchAgent* to query external evidence (line 3). Finally, the multi-run outputs are collected into *Runs* and consolidated using the *Aggregate* function to produce the robust attribute completions R_m.

5. Experiments

This section details the tests we ran and the lessons learned, starting with the presentation of the dataset used and describing our implementation.

5.1. Datasets DOLAP contents BEING UPDATED BY LEO[Vero]

@Leo: Mention why we choose these datasets (their diversity in characteristics). Add a table comparing sizes and co.[Vero]

To evaluate our approach,

5.1.1. The Airport dataset

We built a property graph database on air routes and trade, comprising three node types: `Airport` nodes with geographic coordinates and ICAO codes; `City` nodes with population (in millions) and pollution indices; and `Country` nodes characterized by numerical indicators such as GDP and birth rate. Edges represent different relationships: `FLOW` captures passenger exchanges between airports with property `nPAX`, `ROUTE_TO` represents flights between airports, and `TRADE` models trade flows between countries. We built a Neo4j database instance from multiple Kaggle and Eurostat datasets. An ETL pipeline was implemented to extract, clean, normalize, and integrate the data, ensuring consistency and integrity. The data were enriched with computed properties (e.g., distances between airports and between airports and city centers). Flight prices and durations were collected via web scraping using the FastFlights[2] library. This database exemplifies a dataset rich in numerical properties but poor in textual information. Since textual data are harder to obtain, this setting is well suited for data augmentation using external LLM knowledge. For instance, no dataset explicitly identifies airline hub airports, but this can be easily obtained from external knowledge sources or web searches.

Table 2 reports the difficulty scores computed over the Airports dataset for Koyuk Alfred Adams Airport (PAKK), located in the United States, and Kaadedhdhoo Airport (VRMT), located in the Maldives. The results show that tasks involving properties classified as easy to find, such as an airport's city or altitude, yield lower difficulty scores. In contrast, even when similarly easy properties are used in the H and G sets, less popular airports, such as VRMT, exhibit higher difficulty scores, reflecting the increased challenge of retrieving reliable information for less prominent entities.

[2] https://aweirddev.github.io/flights/

Airport	Task	Specificity	Difficulty
PAKK	$H = \{'runways', 'iata', 'runwaysLength'\}$	4.37	16.72
	$G = \{'runwaysWidth', 'commercialFlights'\}$	12.4	
PAKK	$H = \{city', 'longitude', 'source'\}$	3.29	7.77
	$G = \{'runwaysLength', 'runways', 'runwaysWidth'\}$	4.48	
VRMT	$H = \{'runways', 'iata', 'commercial_flights'\}$	22.16	30.65
	$G = \{city', 'tz_database_timezone', 'altitude'\}$	8.14	
VRMT	$H = \{'icao', 'timezone', 'city'\}$	8.49	16.96
	$G = \{dst', 'type'\}$	8.47	

Table 2
Tasks and their difficulty from the Airports database.

Model	F1 Score	Recall	Context (Y/N)	Effort	Mean EIS
Claude Sonnet 4.5	0.8	0.66	Y	414	0.0076
Claude Haiku 4.5	0.54	0.37	Y	103	0.0002
GPT-5	0.67	0.5	N	377	0.0581
GPT-5 Mini	0.57	0.4	N	325	0.0076
Gemini 2.5 Pro	0.65	0.48	Y	468	0.8042
Gemini 2.5 Flash	0.61	0.44	Y	468	0.0611

Table 3
Best accuracy of each model

5.1.2. The Football dataset
5.1.3. The Music dataset

5.2. Implementation DOLAP contents TO UPDATE[Vero]

We implemented the protocol described in Section **??** using Python. The code used is publicly available at GitHub[3 Felipe & Cristina]. The centrality function $Cent()$ was computed using PageRank, while the property popularity function $pop(p)$ was estimated from the Airports dataset. The context function $context()$ was computed following the approach described in [1]. Based on these quantities, tasks were generated as described in Algorithm 2. Model inference was performed using the official APIs of each provider, leveraging batch inference. Finally, we applied the following two post-processing steps to the collected outputs.

Since LLMs are inherently non-deterministic, we apply a voting function to obtain a single representative result per configuration. This function relies on data integration techniques tailored to the output data type [27]. For textual attributes, we select the most frequent value while for numerical attributes, we compute the mean. We also introduce a tolerance function to account for cases where model outputs are semantically correct but do not exactly match the ground truth. For example, a ground-truth value such as *"Guarulhos–São Paulo Airport"* may be returned as *"Guarulhos/São Paulo Airport"*. To handle such variations, we apply different similarity criteria depending on the data type. For textual values, we use a overlap coefficient with a threshold of 75% similarity. We employed the overlap coefficient because it measures containment of the ground-truth tokens rather than penalizing additional terms.[Felipe & Cristina] For integer values, we rely on a relative-error–based similarity measure. For geospatial attributes (latitude and longitude), we compute the Haversine distance and consider predictions within a 10 km radius as correct. Any model output satisfying the corresponding threshold is counted as a correct answer.

5.3. Lessons learned DOLAP contents TO UPDATE[Vero]

Difficulty vs Accuracy Table 3 reports the best accuracy for each model with the corresponding effort and the EIS normalized with log minmax and bound to 0-1. It should be noted that precision is steadily equals to 1 for all models and situations (with or without context), since models are explicitly asked to produce results only when they are confident. This means that accuracy equals recall. It can be seen that all models struggle to achieve a F1 score greater than 0.8. We note that recall may be impacted by our prompting policy: LLM answer only when they are confident that the values are trustworthy, and we only count answers that strictly follow the requested CSV structure. Modifying this policy is part of our future work.[Patrick]

[3] https://github.com/felipeVsc/llmaugmentation

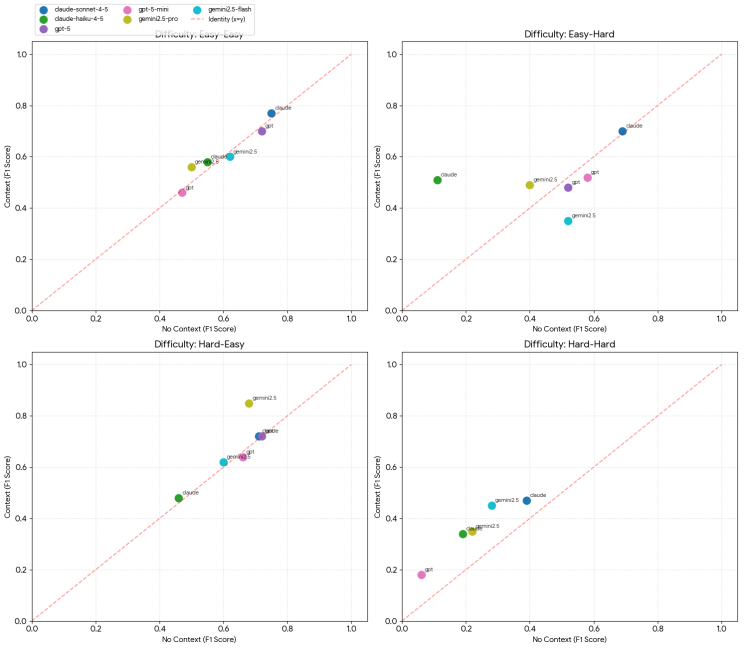


Figure 3: Accuracy vs Difficulty (of H and G)

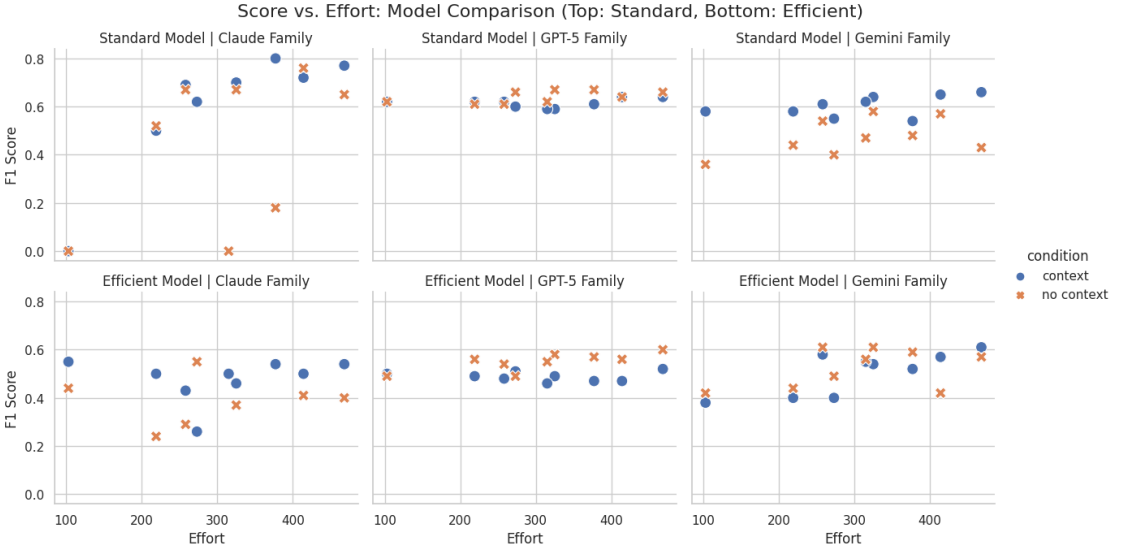


Figure 4: Accuracy vs Effort

Figure 3 plots accuracy vs difficulty of H and G (computed on the ground truth)[Patrick] for all the models tested, with and without context (same definitions as above). Difficulty of H and G is given as: Easy (property popularity in the fourth quartile) and Hard (property popularity in the first quartile). It can be seen that accuracy decreases with difficulty, as expected, and that context is useful when difficulty is high (hard-hard quadrant, where accuracy is systematically above the diagonal).

Effort vs Accuracy Figure 4 plots accuracy vs prompting effort for all the models tested, with and without context. Without context means that all properties in sets H and G come from node n while with context means that properties

in G are randomly chosen from the node n and its context. It can be seen that achieving a better accuracy with more human effort is very model dependent. This is related to the number of extra tokens generated by models with different prompting strategies (not reported due to lack of space).

6. Conclusion DOLAP contents TO UPDATE[Vero]

This paper proposes an experimental protocol to evaluate LLM-based data augmentation tasks of various difficulties in property graphs, in terms of accuracy, prompting effort, and environmental impact. Our preliminary results show that most of the models tested struggle with such tasks and that context is useful for difficult tasks, with variations in the model and prompting strategy used. Our current work includes the evaluation of our approach over various datasets (for instance from the Open Graph Benchmark[4]) and a comparison to baseline data augmentation approaches. On a longer term, we plan to investigate how techniques like RAG or agentic AI could help in achieving better accuracies. [Patrick]
Discuss "domain" difficulty? as future work.[Vero]

References

[1] C. Aguiar, J. Chabin, A. Chanson, M. Halfeld-Ferrari, N. Hiot, N. Labroche, P. Marcel, V. Peralta, and F. Vasconcelos. Extracting node comparison insights for the interactive exploration of property graphs, 2025.

[2] Anthropic. Claude models overview. https://platform.claude.com/docs/en/about-claude/models/overview, 2025.

[3] M. F. Argerich and M. Patiño-Martínez. Measuring and improving the energy efficiency of large language models inference. *IEEE Access*, 12:80194–80207, 2024.

[4] A. Bahi, A. Ourici, and I. Gasmi. Environmental impact of generative and agentic ai: An in-depth analysis and green solutions. *arXiv preprint arXiv:2609.32960*, 2026.

[5] T. Brown, B. Mann, N. Ryder, M. Subbiah, J. D. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell, et al. Language models are few-shot learners. *Neurips*, 33:1877–1901, 2020.

[6] S. K. Card, T. P. Moran, and A. Newell. The keystroke-level model for user performance time with interactive systems. *Communications of the ACM*, 23(7):396–410, 1980.

[7] S. Castelo, R. Rampin, A. S. R. Santos, et al. Auctus: A dataset search engine for data discovery and augmentation. *Proc. VLDB Endow.*, 14(12):2791–2794, 2021.

[8] R. Castro Fernandez, Z. Abedjan, F. Koko, et al. Aurum: A data discovery system. In *ICDE*, 2018.

[9] A. Chapman, E. Simperl, L. Koesten, et al. Dataset search: a survey. *VLDB J.*, 29(1):251–272, 2020.

[10] B. Ding, C. Qin, R. Zhao, et al. Data augmentation using large language models: Data perspectives, learning paradigms and challenges, 2024.

[11] Y. Dong, C. Xiao, T. Nozawa, et al. Deepjoin: Joinable table discovery with pre-trained language models. *Proc. VLDB Endow.*, 16(10):2458–2470, 2023.

[12] C. Dutra Aguiar, J. Chabin, A. Chanson, M. Halfeld Ferrari Alves, N. Hiot, N. Labroche, P. Marcel, V. Peralta, and F. Vasconcelos. Extracting node comparison insights for the interactive exploration of property graphs. *CoRR*, abs/2512.15157, 2025.

[13] A. Dutulescu, S. Ruseti, M. Dascalu, et al. How hard can this question be? an exploratory analysis of features assessing question difficulty using llms. In *EDM*, 2024.

[14] G. Fan, J. Wang, Y. Li, et al. Table discovery in data lakes: State-of-the-art and future directions. In *SIGMOD/PODS*, 2023.

[4] https://ogb.stanford.edu/

[15] J. Freire, G. Fan, B. Feuer, et al. Large language models for data discovery and integration: Challenges and opportunities. *IEEE Data Eng. Bull.*, 49(1):3–31, 2025.

[16] M. Gabburo, N. P. Jedema, S. Garg, et al. Measuring retrieval complexity in question answering systems. In *Findings of the ACL*, pages 14636–14650. Association for Computational Linguistics, 2024.

[17] M. Geva, J. Bastings, K. Filippova, and A. Globerson. Dissecting recall of factual associations in auto-regressive language models, 2023.

[18] Z. He, S. Naphade, and T. K. Huang. Prompting in the dark: Assessing human performance in prompt engineering for data labeling when gold labels are absent. In *CHI*, pages 1195:1–1195:33. ACM, 2025.

[19] A. Helal, M. Helali, K. Ammar, et al. A demonstration of kglac: A data discovery and enrichment platform for data science. *Proc. VLDB Endow.*, 14(12):2675–2678, 2021.

[20] Y. Huang, Y. Gao, and C. Ren. A survey of data augmentation in named entity recognition. *Neurocomputing*, 651:130856, 2025.

[21] Z. Ji and M. Jiang. A systematic review of electricity demand for large language models: evaluations, challenges, and solutions. *Renewable and Sustainable Energy Reviews*, 225(C):None, None 2026.

[22] A. Khatiwada, H. Kokel, I. Abdelaziz, et al. Tabsketchfm: Sketch-based tabular representation learning for data discovery over data lakes. In *ICDE*, 2025.

[23] O. Kim. Detail matters: Measuring the impact of prompt specificity on reasoning in large language models, 2025.

[24] T. Kwa, B. West, J. Becker, et al. Measuring AI ability to complete long tasks. *CoRR*, abs/2503.14499, 2025.

[25] V. D. Martino, M. A. Zadenoori, X. Franch, et al. Green prompt engineering: Investigating the energy impact of prompt design in software engineering, 2025.

[26] F. Nagle and D. Yue. The latent role of open models in the ai economy. *Available at SSRN 5767103*, 2025.

[27] F. Naumann and J. Bleiholder. Conflict handling strategies in an integrated information system. In *Proceedings of the WWW Workshop in Information integration on the Web (IIWEB)*, 2006.

[28] N. W. Paton, J. Chen, and Z. Wu. Dataset discovery and exploration: A survey. *ACM Comput. Surv.*, 56(4):102:1–102:37, 2024.

[29] F. Petroni, T. Rocktäschel, P. Lewis, A. Bakhtin, Y. Wu, A. H. Miller, and S. Riedel. Language models as knowledge bases?, 2019.

[30] S. Rincé and A. Banse. Ecologits: Evaluating the environmental impacts of generative ai. *Journal of Open Source Software*, 10(111):7471, 2025.

[31] P. Sahoo, A. K. Singh, S. Saha, V. Jain, S. Mondal, and A. Chadha. A systematic survey of prompt engineering in large language models: Techniques and applications. *arXiv preprint arXiv:2402.07927*, 2024.

[32] K. Trinh, S. Seidenberger, R. Wijewickrama, et al. A picture is worth a thousand prompts? efficacy of iterative human-driven prompt refinement in image regeneration tasks. In *IJCAI*, pages 10189–10197. ijcai.org, 2025.

[33] F. Vasconcelos, C. Aguiar, A. Chanson, M. Halfeld-Ferrari, P. Marcel, and V. Peralta. On the efficacy of using llms for context driven entity augmentation in property graphs. In *DOLAP'2026 (EDBT/ICDT workshop)*, Tampere, Finland, 2026.

[34] Q. Wang and R. C. Fernandez. Solo: Data discovery using natural language questions via A self-supervised approach. *Proc. ACM Manag. Data*, 1(4):262:1–262:27, 2023.

[35] W. Wei, X. Ren, J. Tang, et al. Llmrec: Large language models with graph augmentation for recommendation, 2024.

[36] C. White, S. Dooley, M. Roberts, et al. Livebench: A challenging, contamination-limited LLM benchmark. In *The Thirteenth International Conference on Learning Representations*, 2025.

[37] J. Wu, J. Zhu, Y. Liu, M. Xu, and Y. Jin. Agentic reasoning: A streamlined framework for enhancing LLM reasoning with agentic tools. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 28489–28503. Association for Computational Linguistics, July 2025.

[38] Y. Xi, J. Lin, Y. Xiao, Z. Zhou, R. Shan, T. Gao, J. Zhu, W. Liu, Y. Yu, and W. Zhang. A survey of large language model-based search agents. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 8244–8279. Association for Computational Linguistics, 2026.

[39] H. Zhang, Y. Liu, W. Hung, et al. Autoddg: Automated dataset description generation using large language models. *CoRR*, abs/2502.01050, 2025.

[40] M. J. Q. Zhang and E. Choi. Clarify when necessary: Resolving ambiguity through interaction with lms. In *Findings of the Association for Computational Linguistics: NAACL 2025*, pages 5526–5543. Association for Computational Linguistics, 2025.

[41] W. Zhang, Y. Li, Y. Bei, J. Luo, G. Wan, L. Yang, C. Xie, Y. Yang, W.-C. Huang, C. Miao, H. P. Zou, X. Luo, Y. Zhao, Y. Chen, C. Chan, P. Zhou, X. Zhang, C. Zhang, J. Shang, M. Zhang, Y. Song, I. King, and P. S. Yu. From web search towards agentic deep research: Incentivizing search with reasoning agents, 2025.

[42] J. Zhou, C. Xie, S. Gong, et al. Data augmentation on graphs: A technical survey. *ACM Comput. Surv.*, 57(11):274:1–274:34, 2025.

[43] Y. Zhu, D. Liu, Z. Lin, et al. The LLM already knows: Estimating llm-perceived question difficulty via hidden representations. *CoRR*, abs/2509.12886, 2025.